

# Pruebas de Seguridad — Módulo Bolsillos y Depósito (FuBanking)

|                |                                                                                   |
|----------------|-----------------------------------------------------------------------------------|
| Asignatura     | Validación y Verificación de Software                                             |
| Proyecto       | FuBanking — Banco digital                                                         |
| Alcance        | Bolsillos (crear, consultar, actualizar, eliminar, transferir) + Depositar dinero |
| Tipo de prueba | Seguridad (security testing) convertida en regresión automatizada                 |
| Rama           | tests/bolsillos-calidad (desde dev)                                               |
| Fecha          | 2026-10-08                                                                        |

## 1. Resumen ejecutivo

Se diseñó y ejecutó una suite de **158 pruebas de seguridad**: 154 en el backend y 4 en el frontend. Cubren las seis funcionalidades frente a cinco familias de amenazas: autenticación, autorización, validación de entrada, neutralización de salida y endurecimiento de la aplicación.

La suite confirmó **8 hallazgos**. El ciclo fue *documentar* → *refactorizar*: primero cada hallazgo quedó como prueba que afirma el comportamiento seguro y falla (**it.fails**); después se corrigieron los cuatro de mayor impacto y sus pruebas pasaron a ser normales.

|                                                                       | Antes del refactor       | Después del refactor     |
|-----------------------------------------------------------------------|--------------------------|--------------------------|
| Pruebas de seguridad del backend que pasan                            | 117                      | <b>144</b>               |
| Pruebas que documentan una vulnerabilidad abierta ( <b>it.fails</b> ) | 37                       | <b>10</b>                |
| Hallazgos abiertos                                                    | 8                        | <b>4</b>                 |
| Suite completa del backend                                            | 772 + 42 <b>it.fails</b> | <b>801 + 13 it.fails</b> |
| Cobertura de líneas del backend                                       | —                        | <b>99,6 %</b>            |

**Hallazgo más grave (corregido)**: el token temporal que se entrega tras escribir la contraseña, *antes* de validar el código OTP, servía como token de acceso completo. Quien conociera la contraseña de un usuario podía operar sus bolsillos y depositar **sin el segundo factor** (bypass de 2FA). El token de recuperación de contraseña tenía el mismo problema.

## 2. Sistema bajo prueba

### 2.1 Superficie de ataque

| # | Funcionalidad       | Endpoint                               | Entrada controlada por el cliente |
|---|---------------------|----------------------------------------|-----------------------------------|
| 1 | Crear bolsillo      | POST /api/v1/pockets                   | accountId, name, amount           |
| 2 | Consultar bolsillos | GET /api/v1/pockets/account/:accountId | accountId (URL)                   |
| 3 | Actualizar bolsillo | PATCH /api/v1/pockets/:pocketId        | pocketId (URL), name, amount      |

| # | Funcionalidad              | Endpoint                          | Entrada controlada por el cliente |
|---|----------------------------|-----------------------------------|-----------------------------------|
| 4 | Eliminar bolsillo          | DELETE /api/v1/pockets/:pocketId  | pocketId (URL)                    |
| 5 | Transferir entre bolsillos | POST /api/v1/pockets/transfer     | fromPocketId, toPocketId, amount  |
| 6 | Depositar dinero           | POST /api/v1/accounts/:id/deposit | id (URL), amount, description     |

Las seis exigen **Authorization: Bearer <JWT>**. La identidad del usuario sale **solo** del token.

## 2.2 Activos y límites de confianza

| Activo                                                 | Por qué importa                                         |
|--------------------------------------------------------|---------------------------------------------------------|
| Saldo disponible de la cuenta y monto de cada bolsillo | Integridad del dinero del cliente                       |
| Identidad del usuario (JWT)                            | Todo el control de acceso depende de ella               |
| Segundo factor (OTP por correo)                        | Protege la cuenta aunque se filtre la contraseña        |
| Nombre del bolsillo                                    | Texto libre que se muestra en el navegador (riesgo XSS) |

Navegador (Next.js) —HTTPS + JWT→ API Express → casos de uso → Supabase  
 [no confiable] | authMiddleware / adminMiddleware  
 | validadores zod · errorHandler · helmet · CORS  
 límite de confianza

Todo lo que llega en la petición (cabeceras, URL, body) es **no confiable** hasta que lo validan **authMiddleware**, los validadores y las reglas de dominio.

## 3. Modelo de amenazas

### 3.1 STRIDE aplicado al módulo

| Categoría                | Amenaza concreta en Bolsillos/Depósito                                                   | Regla que la cubre                                                          |
|--------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Spoofing (suplantación)  | Usar un token falsificado, vencido, sin firma o de otro propósito                        | SEC-AUTH                                                                    |
| Tampering (manipulación) | Alterar el payload del JWT; colar <b>userId/accountId/id</b> en el body; montos ambiguos | SEC-AUTH, SEC-AUTHZ, SEC-IN-01/02                                           |
| Repudiation              | Operaciones sin rastro                                                                   | Fuera de alcance (se notifica cada operación; no hay bitácora de auditoría) |
| Information disclosure   | Ver bolsillos ajenos; errores con stack trace o rutas internas                           | SEC-AUTHZ, SEC-HD-04                                                        |
| Denial of service        | Ráfagas sin límite; cuerpos gigantes o malformados                                       | SEC-HD-03, SEC-HD-05                                                        |
| Elevation of privilege   | Operar recursos de otro usuario; usar un token temporal como acceso                      | SEC-AUTHZ, SEC-AUTH                                                         |

### 3.2 Correspondencia con OWASP API Security Top 10 (2023)

| OWASP | Riesgo                                                       | Cubierto por                                                       |
|-------|--------------------------------------------------------------|--------------------------------------------------------------------|
| API1  | Broken Object Level Authorization (BOLA / IDOR)              | SEC-AUTHZ (IDOR en las 6 operaciones)                              |
| API2  | Broken Authentication                                        | SEC-AUTH (8 variantes × 6 operaciones + tokens de propósito único) |
| API3  | Broken Object Property Level Authorization (mass assignment) | SEC-AUTHZ (campos colados en el body)                              |
| API4  | Unrestricted Resource Consumption                            | SEC-HD-03 (body), SEC-HD-05 (rate limiting)                        |
| API6  | Unrestricted Access to Sensitive Business Flows              | SEC-AUTHZ (mover dinero entre cuentas)                             |
| API8  | Security Misconfiguration                                    | SEC-HD-01/02/04 (helmet, CORS, errores)                            |
| API10 | Unsafe Consumption / inyección                               | SEC-IN-03/04/05, SEC-OUT-01                                        |

## 4. Metodología

### 4.1 Modelo del curso

Se siguió el ejemplo `security-testing` del curso: **cada regla de seguridad se convierte en una prueba de regresión**, con tablas negativas parametrizadas e identificadores legibles. En el ejemplo, la política de contraseñas (`validar_password_segura`) y el escape de HTML (`sanitizar_entrada`) se prueban con `pytest.mark.parametrize`. Aquí se aplica lo mismo en **Vitest** (estándar del equipo) con `it.each` / `describe.each`, sobre las seis funcionalidades:

| Ejemplo del curso                                          | Equivalente en FuBanking                                                   |
|------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>validar_password_segura</code> (política de entrada) | SEC-IN-01: política de montos (tabla de 4 válidos y 12 inválidos)          |
| <code>sanitizar_entrada</code> (escape de HTML)            | SEC-IN-04 (la API guarda literal) + SEC-OUT-01 (React pinta como texto)    |
| Tablas <code>parametrize</code> con <code>ids</code>       | <code>it.each</code> / <code>describe.each</code> con descripción por caso |

### 4.2 Niveles de prueba

| Nivel                                               | Qué ejecuta                                                                                                     | Para qué                                            |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| API sobre app en memoria                            | <code>authMiddleware</code> , validadores, controladores y casos de uso <b>reales</b> ; persistencia en memoria | Autenticación, autorización, entrada, rate limiting |
| App real de producción ( <code>createApp()</code> ) | helmet, CORS, <code>express.json({ limit: '10kb' })</code> , <code>errorHandler</code>                          | Endurecimiento (sin llegar a Supabase)              |
| Componente React (jsdom)                            | <code>PocketsClient</code> con el servicio simulado                                                             | Neutralización de salida (XSS)                      |

Los tokens temporales de 2FA y de recuperación **no se fabricaron a mano**: los emite el código real (`GenerateTwoFactorCode`, `RequestPasswordReset`). Así la prueba reproduce lo que un atacante obtendría del sistema.

### 4.3 Ciclo documentar → refactorizar

1. **Documentar.** Cada prueba afirma el comportamiento seguro. Si falla por una vulnerabilidad real, se marca `it.fails` con el ID del hallazgo: la suite queda verde y el hallazgo visible.
2. **Verificar el motivo.** Cada `it.fails` se ejecutó como prueba normal para confirmar que falla **justo en la aserción de seguridad** y no por un error del propio test (`02-motivo-de-cada-hallazgo.txt`).
3. **Verificar los controles.** Las pruebas que pasan se sometieron a mutaciones: se eliminó a propósito el control y la prueba debía fallar (`03-sensibilidad-de-los-controles.txt`).
4. **Refactorizar.** Tras corregir, Vitest reporta como fallo cada `it.fails` que ahora pasa: es la evidencia de que la vulnerabilidad se cerró (`04-refactor-cierra-los-hallazgos.txt`). Esas pruebas pasan a ser normales y quedan como regresión permanente.

## 5. Reglas y casos de prueba

| Regla      | Descripción                                                                          | Archivo                                        | Casos      |
|------------|--------------------------------------------------------------------------------------|------------------------------------------------|------------|
| SEC-AUTH   | Solo un token de <b>acceso</b> válido permite operar                                 | <code>autenticacion.security.test.ts</code>    | 69         |
| SEC-AUTHZ  | Un usuario solo opera sobre sus recursos; la identidad sale del token                | <code>autorizacion.security.test.ts</code>     | 15         |
| SEC-IN-01  | Montos estrictos: número JSON o texto decimal, sin conversión forzada                | <code>entrada.security.test.ts</code>          | 32         |
| SEC-IN-02  | Montos dentro del rango entero seguro                                                | <code>entrada.security.test.ts</code>          | 2          |
| SEC-IN-03  | Identificadores maliciosos en la URL no rompen nada ni tocan datos                   | <code>entrada.security.test.ts</code>          | 11         |
| SEC-IN-04  | HTML, SQL, NoSQL o plantillas en el nombre se tratan como dato literal               | <code>entrada.security.test.ts</code>          | 7          |
| SEC-IN-05  | <code>__proto__</code> / <code>constructor</code> no contaminan objetos del servidor | <code>entrada.security.test.ts</code>          | 1          |
| SEC-HD-01  | Cabeceras de seguridad (helmet) en respuestas normales y de error                    | <code>endurecimiento.security.test.ts</code>   | 3          |
| SEC-HD-02  | CORS solo para el origen del frontend                                                | <code>endurecimiento.security.test.ts</code>   | 2          |
| SEC-HD-03  | Body malformado o excesivo es error del cliente (4xx)                                | <code>endurecimiento.security.test.ts</code>   | 2          |
| SEC-HD-04  | Los errores no exponen detalles internos                                             | <code>endurecimiento.security.test.ts</code>   | 4          |
| SEC-HD-05  | Límite de frecuencia por usuario (rate limiting)                                     | <code>endurecimiento.security.test.ts</code>   | 6          |
| SEC-OUT-01 | Un nombre malicioso se muestra como texto en el navegador                            | <code>frontend/.../pockets.xss.test.tsx</code> | 4          |
|            | <b>Total</b>                                                                         |                                                | <b>158</b> |

Detalle de las tablas principales:

- **SEC-AUTH**, por cada una de las 6 operaciones: sin cabecera, esquema **Basic**, **Bearer** vacío, token malformado, firmado con otro secreto, vencido, algoritmo **none** (sin firma) y payload alterado para suplantar al titular. Siempre 401, con el código exacto y **sin efectos** (saldo, bolsillos y notificaciones intactos). Incluye un control positivo con un token legítimo y los tokens de propósito único (2FA y recuperación), también contra **adminMiddleware**.
- **SEC-AUTHZ**: IDOR en las 6 operaciones (el intruso usa ids del titular → 403 y ninguna cuenta cambia); tres intentos de mover dinero entre cuentas de distintos dueños (→ 400, no se mueve dinero); mass assignment al crear, actualizar, depositar y transferir.
- **SEC-IN-01**: 4 montos válidos (**1500**, **1500.5**, **"1500"**, **"1500.50"**) y 12 inválidos (**true**, **false**, **null**, **[]**, **[1500]**, **{...}**, **" "**, **" "**, **"abc"**, **"0x10"**, **"1e3"**, **-1**), aplicados a crear bolsillo y a depositar.

## 6. Hallazgos

### 6.1 Resumen

Severidad cualitativa del probador; el vector CVSS 3.1 es una **estimación** para comparar hallazgos, no una calificación oficial.

| ID     | Hallazgo                                                                            | Severidad   | CWE     | OWASP | Casos | Estado      |
|--------|-------------------------------------------------------------------------------------|-------------|---------|-------|-------|-------------|
| SEC-01 | El token temporal de 2FA sirve como token de acceso (bypass de 2FA)                 | Alta (7.4)  | CWE-287 | API2  | 7     | ✅ Corregido |
| SEC-02 | El token de recuperación de contraseña sirve como token de acceso                   | Alta (7.4)  | CWE-287 | API2  | 7     | ✅ Corregido |
| SEC-03 | Type juggling del monto al crear bolsillo ( <b>z.coerce.number</b> ) — incluye D-02 | Media (4.3) | CWE-20  | API8  | 9     | ✅ Corregido |
| SEC-04 | Type juggling del monto al depositar ( <b>Number(amount)</b> ) — incluye D-08       | Media (4.3) | CWE-20  | API8  | 4     | ✅ Corregido |
| SEC-05 | Se aceptan montos fuera del rango entero seguro (pérdida de precisión)              | Media (4.3) | CWE-681 | API8  | 2     | ⌚ Abierto   |
| SEC-06 | JSON malformado responde 500 en vez de 400                                          | Baja        | CWE-755 | API8  | 1     | ⌚ Abierto   |
| SEC-07 | Body de más de 10 KB responde 500 en vez de 413                                     | Baja        | CWE-755 | API4  | 1     | ⌚ Abierto   |
| SEC-08 | Sin límite de frecuencia en Bolsillos y depósito                                    | Media (4.3) | CWE-770 | API4  | 6     | ⌚ Abierto   |

Vectores CVSS 3.1 estimados:

| Hallazgos | Vector | Puntaje |
|-----------|--------|---------|
|-----------|--------|---------|

| Hallazgos              | Vector                              | Puntaje |
|------------------------|-------------------------------------|---------|
| SEC-01, SEC-02         | AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N | 7.4     |
| SEC-03, SEC-04, SEC-05 | AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N | 4.3     |
| SEC-08                 | AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:L | 4.3     |

SEC-01 y SEC-02 tienen complejidad alta (AC:H) porque el atacante necesita antes la contraseña o el enlace del correo de la víctima.

## 6.2 SEC-01 — Bypass del segundo factor (2FA)

### Escenario de ataque.

1. El atacante conoce la contraseña de la víctima (filtración, reutilización, phishing).
2. Hace `POST /auth/login`; como la víctima tiene 2FA, recibe un `temporaryToken` y el código OTP se envía **al correo de la víctima**, que el atacante no ve.
3. Usa ese `temporaryToken` como `Authorization: Bearer ...` en cualquier endpoint protegido.
4. **Resultado observado:** crear (201), consultar, actualizar, eliminar, transferir y depositar (200), e incluso las rutas de administrador (200) si la víctima es admin.

**Causa raíz.** `GenerateTwoFactorCode` firmaba el token temporal con el mismo secreto y la misma forma que un token de acceso (`{ userId, email, rememberMe }`, sin `type`), y `authMiddleware` / `adminMiddleware` aceptaban cualquier JWT con firma válida, sin distinguir su **propósito** (*token confusion*).

**Impacto.** Anula el segundo factor: confidencialidad e integridad totales sobre el dinero de la víctima durante la vida del token (10 minutos, renovable con `/2fa/resend`).

## 6.3 SEC-02 — El enlace de recuperación da acceso a la API

El token de `/auth/forgot-password` (`type: 'reset'`, 5 minutos) viaja en un **enlace por correo**. `authMiddleware` lo aceptaba como token de acceso: quien obtuviera el enlace (correo comprometido, reenvío, historial o registros de un proxy) operaba la cuenta **sin contraseña y sin 2FA**. Misma causa raíz que SEC-01.

## 6.4 SEC-03 y SEC-04 — Type juggling en los montos

`z.coerce.number()` (crear) y `Number(amount)` (depositar) convierten a la fuerza cualquier valor:

| Valor enviado                         | Crear bolsillo (antes) | Depositar (antes)       |
|---------------------------------------|------------------------|-------------------------|
| <code>true</code>                     | bolsillo de \$1        | depósito de \$1         |
| <code>null, [], "", " ", false</code> | bolsillo de \$0        | rechazado (0 no es > 0) |
| <code>[1500]</code>                   | bolsillo de \$1.500    | depósito de \$1.500     |
| <code>"0x10"</code>                   | bolsillo de \$16       | depósito de \$16        |
| <code>"1e3"</code>                    | bolsillo de \$1.000    | depósito de \$1.000     |

Un monto que no es un número se interpreta en silencio como otra cantidad. Es una falla de integridad de la validación: los controles de cliente o de un WAF que miren el valor original no coinciden con lo que procesa el servidor.

## 6.5 Hallazgos abiertos

| ID     | Recomendación                                                                                                                                                                     | Esfuerzo |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| SEC-05 | Rechazar montos que no sean <code>Number.isSafeInteger</code> (o modelar el dinero en centavos enteros / <code>bigint</code> / decimal) y fijar un tope de negocio por operación. | Bajo     |
| SEC-06 | En <code>errorHandler</code> , mapear <code>SyntaxError</code> con <code>type === 'entity.parse.failed'</code> a <b>400</b> .                                                     | Bajo     |
| SEC-07 | Mapear <code>type === 'entity.too.large'</code> a <b>413</b> . Ambos evitan además registrar errores del cliente como "Unexpected error".                                         | Bajo     |
| SEC-08 | Aplicar <code>createRateLimiter</code> (el mismo de Créditos y login) a las rutas de Bolsillos y depósito, con clave por usuario.                                                 | Bajo     |

## 6.6 Observaciones (no son vulnerabilidades confirmadas)

- **Enumeración por 403/404.** Un bolsillo ajeno responde 403 y uno inexistente 404, lo que revela si un id existe. Los ids son UUID v4 (no adivinables), así que el riesgo es bajo.
- **Dinero en coma flotante.** Los montos son `number` de JavaScript; para dinero se recomienda enteros en la unidad mínima.
- **VerifyTwoFactorCode y ResendTwoFactorCode** no exigen `type === '2fa'` en el token que reciben. Con el refactor un token de acceso ya no sirve de atajo hacia el login, pero exigerlo cerraría el último caso de *token confusion*.
- **WithdrawMoney** (módulo de otro compañero) usa el mismo patrón `Number(amount)` que tenía SEC-04; se recomienda reutilizar `parseStrictAmount`.

## 7. Corrección aplicada (refactor)

### 7.1 SEC-01 y SEC-02 — Tokens con propósito y verificación por lista blanca

```
// presentation/middlewares/accessToken.ts (nuevo)
export function isAccessToken(payload: TokenPayload): boolean {
  return payload.type === undefined || payload.type === 'auth';
}

// authMiddleware.ts y adminMiddleware.ts, tras verificar la firma
if (!isAccessToken(payload)) {
  sendError(res, 'Token inválido', 'TOKEN_INVALID', 401);
  return;
}

// GenerateTwoFactorCode.ts: el token temporal declara su propósito
this.tokenService.generate({ userId, email, rememberMe, type: '2fa' }, { expiresIn: ... });
```

#### Decisiones de diseño.

- **Lista blanca, no lista negra:** solo los tokens sin tipo o `'auth'` dan acceso; cualquier tipo futuro queda rechazado por defecto.

- **Compatibilidad:** los tokens de acceso existentes no tienen `type`, así que nadie pierde la sesión. `/auth/2fa/verify` y `/auth/2fa/resend` reciben el token temporal en el body y **no usan** `authMiddleware`, por lo que el login con 2FA funciona igual. El reset ya exigía `type === 'reset'` en su propio flujo.
- **Un solo criterio:** la misma función protege `authMiddleware` y `adminMiddleware`.

## 7.2 SEC-03 y SEC-04 — Monto estricto compartido

```
// presentation/validators/amount.validators.ts (nuevo)
const DECIMAL_TEXT = /^-?\d+(\.\d+)?$/;
export function parseStrictAmount(raw: unknown): number {
  if (typeof raw === 'number') return raw;
  if (typeof raw === 'string' && DECIMAL_TEXT.test(raw.trim())) return Number(raw.trim());
  return Number.NaN; // el caso de uso lo rechaza como monto inválido
}
export const strictAmount = (msg: string) =>
  z.preprocess((raw) => (typeof raw === 'string' ? parseStrictAmount(raw) : raw),
    z.number({ error: 'El monto debe ser un número' })).min(0, msg));
```

- Crear bolsillo: `amount: strictAmount('El monto del bolsillo no puede ser negativo')`.
- Depositar: `amount: parseStrictAmount(amount)` en lugar de `Number(amount)`.
- **Se conserva el contrato:** los textos decimales ("`150000`") siguen aceptándose y los errores mantienen sus códigos y mensajes (`VALIDATION_ERROR`, `INVALID_AMOUNT`). La regresión lo confirma.

## 7.3 Archivos de producción modificados

| Archivo                                                          | Cambio                                                            |
|------------------------------------------------------------------|-------------------------------------------------------------------|
| <code>presentation/middlewares/accessToken.ts</code>             | Nuevo: <code>isAccessToken</code>                                 |
| <code>presentation/middlewares/authMiddleware.ts</code>          | Rechaza tokens que no son de acceso                               |
| <code>presentation/middlewares/adminMiddleware.ts</code>         | Ídem                                                              |
| <code>application/use-cases/auth/GenerateTwoFactorCode.ts</code> | Token temporal con <code>type: '2fa'</code>                       |
| <code>presentation/validators/amount.validators.ts</code>        | Nuevo: <code>parseStrictAmount</code> , <code>strictAmount</code> |
| <code>presentation/validators/pocket.validators.ts</code>        | <code>createPocketSchema</code> usa <code>strictAmount</code>     |
| <code>presentation/controllers/AccountController.ts</code>       | El depósito usa <code>parseStrictAmount</code>                    |

El cambio toca autenticación (módulo de otro integrante del equipo) y afecta a toda la app. Se ejecutaron **todas** las suites del backend y del frontend sin regresiones; se señala en el PR.

## 8. Resultados y evidencia

### 8.1 Antes y después

| Suite                                        | Antes                                                                                                              | Después                                                                                                              |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Security de Bolsillos (backend)              | 117  + 37 <code>it.fails</code> | 144  + 10 <code>it.fails</code> |
| Security de Bolsillos (frontend, SEC-OUT-01) | 4                               | 4                                |

| Suite                                 | Antes                                                                                                      | Después                                                                                                                              |
|---------------------------------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Regresión de Bolsillos                | 193  + 5 <b>it.fails</b>  | 195  + 3 <b>it.fails</b> (D-02 y D-08 corregidos) |
| Backend completo                      | 772  + 42 <b>it.fails</b> | 801  + 13 <b>it.fails</b>                         |
| Frontend completo                     | 249                       | 249                                               |
| Cobertura de líneas / ramas (backend) | —                                                                                                          | 99,6 % / 99,2 %                                                                                                                      |
| Errores de <b>tsc</b>                 | 0                                                                                                          | 0                                                                                                                                    |

## 8.2 El refactor cierra exactamente lo documentado

Al corregir el código **sin tocar las pruebas**, Vitest reportó **29 it.fails** que ahora pasaban: los 7 de SEC-01, los 7 de SEC-02, los 9 de SEC-03, los 4 de SEC-04 y los dos de regresión (D-02 y D-08). Ninguna otra prueba cambió (772 siguieron en verde). Después se convirtieron en pruebas normales.

## 8.3 Sensibilidad de los controles que pasan

| Mutación introducida (y revertida)                                                            | Resultado                                                                          |
|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>DeletePocket</code> sin <code>account.assertBelongsTo</code> (sin control de propiedad) | Falla la prueba IDOR: el intruso borra el bolsillo del titular (200 en vez de 403) |
| <code>PocketsClient</code> pinta el nombre con <code>dangerouslySetInnerHTML</code>           | Fallan las 4 pruebas de XSS: el nombre deja de ser texto y se vuelve HTML          |

Confirma que los controles que pasan lo hacen porque el control existe, no por accidente.

## 8.4 Archivos de evidencia (`docs/evidencia-security-bolsillos/`)

| Archivo                                           | Contenido                                                                                                                                                                |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>01-antes-del-refactor.txt</code>            | Suite completa en fase de documentación (117  + 37 <b>it.fails</b> ) y XSS frontend |
| <code>02-motivo-de-cada-hallazgo.txt</code>       | Cada <b>it.fails</b> ejecutado como prueba normal: aserción exacta que falla                                                                                             |
| <code>03-sensibilidad-de-los-controles.txt</code> | Las dos mutaciones de la sección 8.3                                                                                                                                     |
| <code>04-refactor-cierra-los-hallazgos.txt</code> | Corrida tras el refactor sin tocar las pruebas: los 29 cerrados                                                                                                          |
| <code>05-despues-del-refactor.txt</code>          | Suite de seguridad, regresión y backend completo después del refactor                                                                                                    |

## 9. Riesgo residual

| Riesgo                                        | Nivel | Mitigación actual                              | Siguiente paso                                          |
|-----------------------------------------------|-------|------------------------------------------------|---------------------------------------------------------|
| Ráfagas contra Bolsillos/depósito (SEC-08)    | Medio | Autenticación obligatoria                      | Rate limiting por usuario                               |
| Montos fuera del rango seguro (SEC-05)        | Medio | Ninguna                                        | Validar <code>isSafeInteger</code> y tope por operación |
| Errores 500 por cuerpos inválidos (SEC-06/07) | Bajo  | El body sí se rechaza y no se filtran detalles | Mapear a 400/413 en <code>errorHandler</code>           |

| Riesgo                            | Nivel | Mitigación actual   | Siguiente paso               |
|-----------------------------------|-------|---------------------|------------------------------|
| Robo de contraseña sin 2FA activo | Medio | Depende del usuario | Fuera del alcance del módulo |

## 10. Integración continua

- Las pruebas viven en `backend/src/tests/security/pocket/` y `frontend/src/tests/unit/security/`. Entran solas en `npm run test:coverage` del pipeline de Jenkins (backend y frontend), en `npm run test:security` del equipo y en el script propio `npm run test:security:bolsillos`.
- Un `it.fails` **no rompe** el pipeline mientras la vulnerabilidad siga abierta, pero **sí lo rompe en cuanto se corrige** sin actualizar la prueba: el pipeline obliga a mantener la documentación al día.
- Las pruebas corregidas quedan como **regresión de seguridad permanente**: si alguien reintroduce el bypass de 2FA o la conversión forzada de montos, el pipeline falla.

## 11. Cómo ejecutar

```
cd backend
npm run test:security:bolsillos      # 154 casos con detalle (verbose)
npm run test:security                # toda la carpeta de seguridad del equipo
npm run test:coverage                # lo que ejecuta Jenkins

cd ../frontend
npx vitest run src/tests/unit/security # SEC-OUT-01 (XSS)
```

No requiere Supabase ni red: usa repositorios en memoria y la `createApp()` real sin llegar a la BD.

## 12. Archivos entregados

| Archivo                                                                        | Contenido                                                                             |
|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>backend/src/tests/security/pocket/support/attacks.ts</code>              | Tabla de las 6 operaciones, tokens falsificados y tokens reales de 2FA / recuperación |
| <code>backend/src/tests/security/pocket/autenticacion.security.test.ts</code>  | SEC-AUTH                                                                              |
| <code>backend/src/tests/security/pocket/autorizacion.security.test.ts</code>   | SEC-AUTHZ                                                                             |
| <code>backend/src/tests/security/pocket/entrada.security.test.ts</code>        | SEC-IN-01 a SEC-IN-05                                                                 |
| <code>backend/src/tests/security/pocket/endurecimiento.security.test.ts</code> | SEC-HD-01 a SEC-HD-05                                                                 |
| <code>frontend/src/tests/unit/security/pockets.xss.test.tsx</code>             | SEC-OUT-01                                                                            |
| Archivos de producción de la sección 7.3                                       | Corrección de SEC-01 a SEC-04                                                         |
| <code>docs/evidencia-security-bolsillos/</code>                                | Evidencia de la sección 8.4                                                           |

---

## 13. Guion de sustentación

**En 30 segundos.** Tomé el modelo del curso —convertir reglas de seguridad en pruebas de regresión con tablas— y lo apliqué a mis seis funcionalidades en cinco frentes: autenticación, autorización, entrada, salida y endurecimiento. Encontré 8 vulnerabilidades. La más grave permitía saltarse el 2FA con el token temporal del login. Primero las documenté con pruebas que fallan, después corregí las cuatro más importantes y las pruebas pasaron a verde sin romper nada del resto de la aplicación.

**Demo sugerida.**

1. `npm run test:security:bolsillos` → 144  + 10 `it.fails`.
2. Mostrar `isAccessToken` y explicar la lista blanca.
3. Comentar la verificación `if (!isAccessToken(payload))` en `authMiddleware` → fallan las 12 pruebas SEC-01/SEC-02 de las 6 operaciones: el bypass vuelve. Revertir → verde.

**Preguntas probables.**

- **¿Por qué el token temporal servía como acceso?** Tenía la misma firma y forma que uno de acceso, y el middleware no verificaba el propósito del token.
- **¿Por qué lista blanca?** Si mañana se crea otro token de propósito único, queda rechazado sin tocar el middleware.
- **¿El arreglo rompe el login con 2FA?** No: `/2fa/verify` y `/2fa/resend` reciben el token en el body y no pasan por `authMiddleware`. Las suites de autenticación del equipo siguen en verde.
- **¿Qué es `*type juggling*`?** Que el servidor convierta a la fuerza un valor de otro tipo: `Number(true)` es 1 y `Number([1500])` es 1500.
- **¿Por qué la API guarda `<script>` literal en vez de limpiarlo?** El escape depende del contexto de salida (HTML, SQL, URL...), como advierte el ejemplo del curso. La API devuelve JSON y React lo pinta como texto; las dos cosas están probadas.
- **¿Por qué quedan hallazgos abiertos?** Se priorizaron los de mayor impacto; los otros cuatro están documentados, con su prueba lista para ponerse en verde cuando se corrijan.

---

## 14. Conclusiones

- Las seis funcionalidades quedan protegidas por **158 pruebas de seguridad** que se ejecutan en cada build.
- Los controles de **autorización** (IDOR, mass assignment, abuso de flujo) y de **neutralización** (XSS, inyección, prototype pollution) ya eran sólidos y quedaron verificados con mutaciones.
- Se corrigieron un **bypass de 2FA** (SEC-01), el **uso del enlace de recuperación como acceso** (SEC-02) y la **conversión forzada de montos** (SEC-03/04), sin regresiones en el resto de la app.
- Quedan cuatro hallazgos de severidad media o baja, con recomendación concreta y prueba preparada.